

shared_ptr — Խորը ուղեցույց

C++ Smart Pointer presentation

Այս ուղեցույցը մանրամասն բացատրում է `std::shared_ptr`-ը՝ ինչ է, reference counting, control block, `make_shared`, `use_count`, circular reference-ի խնդիր, օրինակներով և հարցազրույցի հարցերով:

1. Ի՞նչ է `shared_ptr`-ը

`std::shared_ptr`-ը smart pointer է **shared ownership**-ով: Մեկ resource-ը կարող է պատկանել մի քանի `shared_ptr`-ի միասին: Resource-ը ազատվում է, երբ վերջին `shared_ptr`-ը ոչնչանում է:

```
#include <memory>

auto p1 = std::make_shared<int>(10);
auto p2 = p1;           // OK -- copy, երկուսն էլ owner
cout << *p1 << *p2;     // 10 10
// resource ազատվում է երբ եվ p1 եվ p2 ոչնչանում
```

2. Reference Counting

`shared_ptr`-ը պահում է **հաշվիչ** (reference count)՝ քանի owner կա:

```

auto p1 = std::make_shared<int>(10);
cout << p1.use_count();      // 1

auto p2 = p1;
cout << p1.use_count();      // 2

{
    auto p3 = p1;
    cout << p1.use_count(); // 3
}
// p3 նշնանում -> count 2

cout << p1.use_count();      // 2

```

```

copy      -> count++
նշնանում  -> count--
count == 0 -> resource DELETE

```

3. Control Block

shared_ptr-ը ներսում ունի **երկու բան**:

1. Pointer -> resource-ին (տվյալ)
2. Pointer -> control block-ին
 - control block:
 - reference count (strong)
 - weak count
 - deleter

```

p1 --+
    +--> [Control Block: strong=2, weak=0] --> [ Resource: 10 ]
p2 --+

```

Ամեն copy -> նույն control block, strong count++. Սա առանձին հիշողություն է (դրա համար make_shared-ն ավելի լավ է՝ մեկ allocation):

4. make_shared (նախընտրելի)

```
auto p = std::make_shared<int>(10);           // 1 allocation (resource + control block միասին)
// vs
std::shared_ptr<int> q(new int(10));           // 2 allocation (ավելի դանդաղ)
```

`make_shared` -ը resource-ը եւ control block-ը մեկ allocation-ով է ստեղծում -> ավելի արագ, exception-safe:

5. Կարևոր մեթոդներ

```
auto p = std::make_shared<int>(10);

*p;           // dereference
p.get();      // raw pointer
p.use_count(); // քանի owner
p.reset();    // count--, եթե վերջինս է -> delete
if (p) { }    // null չէ?
```

6. Circular Reference — ՄԵԾ ԽՆԴԻՐ

`shared_ptr`-ի ամենամեծ վտանգը՝ **circular reference** (երկու object ցույց են տալիս իրար) -> count երբեք 0 չի դառնում -> LEAK:

```
struct Node {
    std::shared_ptr<Node> next;
};

auto a = std::make_shared<Node>();
auto b = std::make_shared<Node>();
a->next = b;    // b-ի count = 2
b->next = a;    // a-ի count = 2
// scope-ի վերջ: a, b ոչնչանում -> count 1 (իրար պահում) -> երբեք 0 -> LEAK
```

```
a --> [Node A] --> b
b --> [Node B] --> a
circular -> count երբեք 0 չի դառնում -> LEAK
```

Լուծումը՝ weak_ptr

```
struct Node {
    std::weak_ptr<Node> next;    // weak -> count չի ավելացնում
};
```

weak_ptr-ը չի ավելացնում strong count-ը -> circular կոտրվում -> չկա leak. (Տես weak_ptr presentation-ը.)

7. Overhead

```
shared_ptr:
- unique_ptr-ից ավելի ծանր (control block, atomic count)
- reference count-ը ATOMIC է (thread-safe count) -> մի քիչ դանդաղ
- 2 pointer-ի չափով (resource + control block)
```

Օգտագործիր shared_ptr միայն երբ իսկապես shared ownership պետք է; այլ դեպքում unique_ptr:

8. Հարցազրույցի հարցեր և պատասխաններ

shared_ptr ինչ է:

Smart pointer shared ownership-ով; մի քանի owner; վերջին owner-ի ոչնչանալիս resource delete (reference counting):

Reference counting ինչպես:

copy -> count++, ոչնչանում -> count--, count==0 -> delete:

Control block ինչ է:

Առանձին block strong count + weak count + deleter-ով: բոլոր copy-ները նույն control block:

make_shared vs shared_ptr(new):

make_shared 1 allocation (արագ, exception-safe); (new) 2 allocation:

Circular reference ինչ է:

Երկու shared_ptr իրար ցույց տալիս -> count երբեք 0 -> leak. Լուծում՝ weak_ptr:

use_count() ինչ է:

Քանի strong owner կա:

shared vs unique overhead:

shared ավելի ծանր (atomic count, control block); unique zero overhead:

Ամփոփում (Cheat Sheet)

shared_ptr = SHARED ownership (մի քանի owner), reference counting

make_shared<T>(args) -> ստեղծում (1 allocation, ՆԱԽՇՆՏՐԵԼԻ)

copy -> count++ | ոչնչանում -> count-- | count==0 -> DELETE

use_count() -> քանի owner

get() / reset() / *p / p->

CONTROL BLOCK: strong count + weak count + deleter (atomic)

CIRCULAR REFERENCE -> count երբեք 0 -> LEAK -> լուծում՝ weak_ptr

overhead: unique-ից ծանր (atomic count) -> օգտագործիր միայն երբ shared պետք է